

Анализ данных с помощью SQL-запросов

Список всех студентов с названиями их групп

QueryQuery History

```
1 SELECT
2     s.student_id,
3     s.last_name || ' ' || s.first_name || ' ' || COALESCE(s.middle_name, '') AS ФИО,
4     g.group_name AS группа,
5     g.specialty AS специальность
6 FROM students s
7 JOIN groups g ON s.group_id = g.group_id
8 ORDER BY g.group_name, s.last_name;
```

Data OutputMessagesNotifications

Showing rows: 1 to 5

	student_id integer	ФИО text	группа character varying (20)	специальность character varying (100)
1	4	Кузнецов Максим Олегов...	ИС-22	Информационные систе...
2	3	Сидорова Анна Игоревна	ИС-22	Информационные систе...
3	1	Иванов Иван Иванович	П-21	Программирование
4	2	Петров Алексей Сергеевич	П-21	Программирование
5	5	Смирнова Елена Андреев...	ЭК-21	Экономика

В PostgreSQL для конкатенации строк используется оператор `||`. В MySQL — функция `CONCAT()`. Функция `COALESCE()` заменяет `NULL` на пустую строку.

Количество студентов в каждой группе

Query

Query History

1

SELECT

2

g.group_name AS группа,

3

COUNT(s.student_id) AS количество_студентов

4

FROM groups g

5

LEFT JOIN students s ON g.group_id = s.group_id

6

GROUP BY g.group_id, g.group_name

7

ORDER BY количество_студентов DESC;

Data Output

Messages

Notifications

≡+

▼

▼

SQL

	группа character varying (20)	количество_студентов bigint
1	П-21	2
2	ИС-22	2
3	ЭК-21	1
4	П-23	0
5	ТЕСТ-01	0

LEFT JOIN используется, чтобы показать группы, в которых может не быть студентов. Если использовать **INNER JOIN**, группы без студентов исчезнут из результата

Расписание занятий с полной информацией

Query

Query History

1

```
SELECT
  l.lesson_date AS дата,
  g.group_name AS группа,
  d.discipline_name AS дисциплина,
  t.last_name || ' ' || t.first_name AS преподаватель,
  l.lesson_type AS вид_занятия
FROM lessons l
JOIN groups g ON l.group_id = g.group_id
JOIN disciplines d ON l.discipline_id = d.discipline_id
JOIN teachers t ON l.teacher_id = t.teacher_id
ORDER BY l.lesson_date, g.group_name;
```

11

Data Output

Messages

Notifications

≡

📄

▼

📋

▼

🗑

🔍

📥

📶

SQL

Showing rows: 1 to 5

🔍

📄

Pag

	дата date	группа character varying (20)	дисциплина character varying (100)	преподаватель text	вид_занятия character varying (30)
1	2026-09-...	ИС-22	Основы проектирования баз данн...	Волков Алексан...	Практика
2	2026-09-...	П-21	Основы проектирования баз данн...	Волков Алексан...	Лекция
3	2026-09-...	П-21	Программирование	Фёдоров Дмитр...	Практика
4	2026-09-...	ИС-22	Информационные технологии	Орлова Мария	Лекция
5	2026-09-...	ЭК-21	Информационные технологии	Орлова Мария	Практика

Нагрузка преподавателей (количество занятий)

Query

Query History

1

```
SELECT
  t.last_name || ' ' || t.first_name AS преподаватель,
  t.department AS кафедра,
  COUNT(l.lesson_id) AS количество_занятий
FROM teachers t
LEFT JOIN lessons l ON t.teacher_id = l.teacher_id
GROUP BY t.teacher_id, t.last_name, t.first_name, t.department
ORDER BY количество_занятий DESC;
```

8

Data Output

Messages

Notifications

≡

📄

▼

📋

▼

🗑

🔍

📥

📶

SQL

Showing

	преподаватель text	кафедра character varying (100)	количество_занятий bigint
1	Орлова Мария	Общеобразовательные дисципли...	2
2	Волков Алексан...	Информационные технологии	2
3	Фёдоров Дмитр...	Программирование	1

Студенты, родившиеся после 2007 года

Query

Query History

1

```
SELECT
    last_name || ' ' || first_name AS ФИО,
    birth_date AS дата_рождения
FROM students
WHERE birth_date > '2007-01-01'
ORDER BY birth_date;
```

Data Output

Messages

Notifications

≡

📄

▼

📋

▼

🗑

📦

⬇

📈

SQL

Showing rows: 1 to 3

📄

Page No: 1

	ФИО text	дата_рождения date
1	Кузнецов Макс...	2007-01-18
2	Иванов Иван	2007-03-15
3	Петров Алексей	2007-07-21

Дисциплины с количеством часов больше 72

Query

Query History

1

```
SELECT
    discipline_name AS дисциплина,
    hours AS часы
FROM disciplines
WHERE hours > 72
ORDER BY hours DESC;
```

Data Output

Messages

Notifications

≡

📄

▼

📋

▼

🗑

📦

⬇

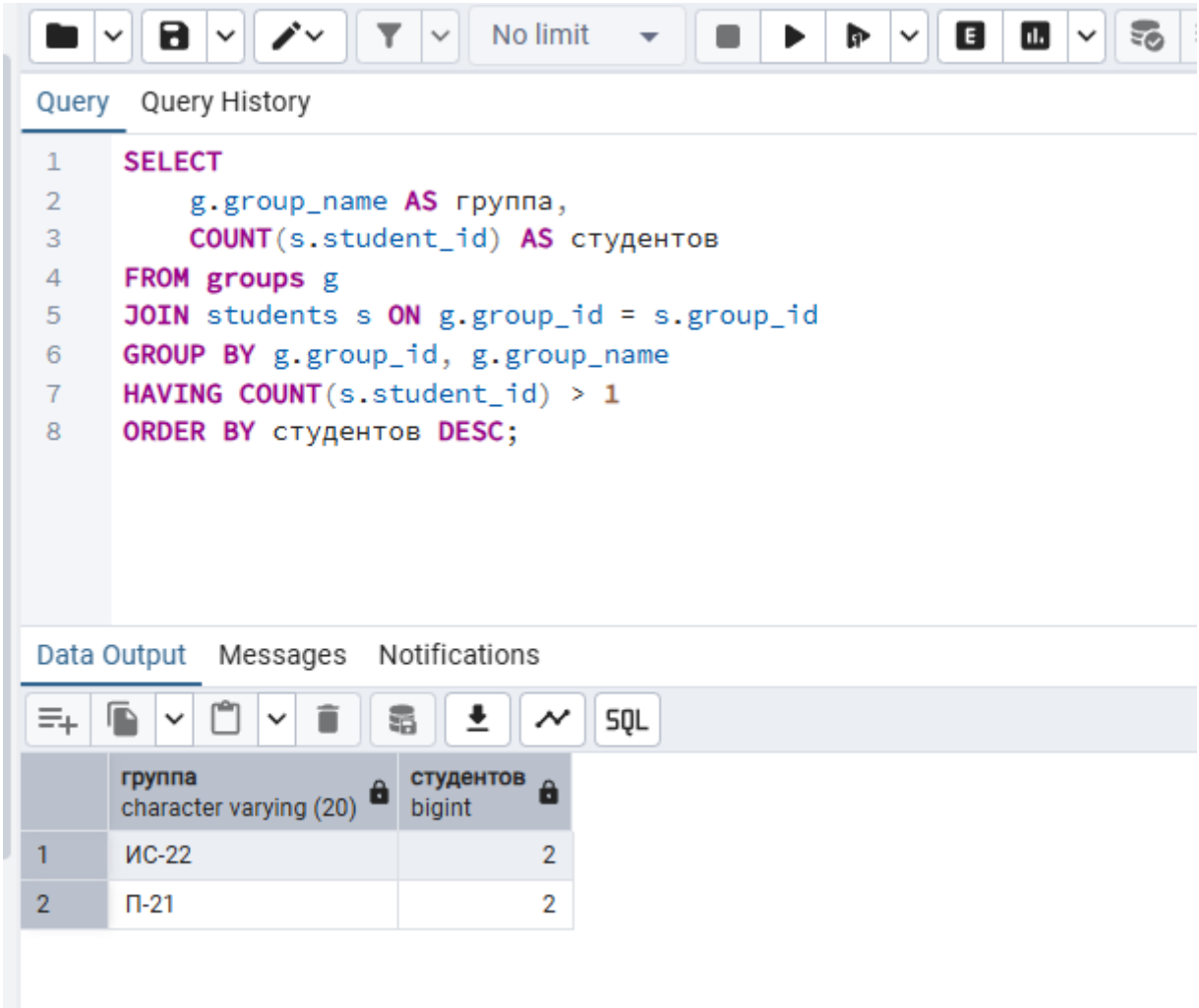
📈

SQL

Sh

	дисциплина character varying (100)	часы integer
1	Программирование	144

Группы, в которых больше одного студента



The screenshot shows a SQL IDE interface. At the top is a toolbar with icons for file operations, filters, and execution. Below the toolbar are tabs for 'Query' and 'Query History'. The 'Query' tab is active, displaying an SQL query. Below the query editor are tabs for 'Data Output', 'Messages', and 'Notifications'. The 'Data Output' tab is active, showing a table with two columns: 'группа' (group) and 'студентов' (students). The table contains two rows of data.

```
1 SELECT
2     g.group_name AS группа,
3     COUNT(s.student_id) AS студентов
4 FROM groups g
5 JOIN students s ON g.group_id = s.group_id
6 GROUP BY g.group_id, g.group_name
7 HAVING COUNT(s.student_id) > 1
8 ORDER BY студентов DESC;
```

	группа character varying (20)	студентов bigint
1	ИС-22	2
2	П-21	2

HAVING используется для фильтрации групп после агрегации, в отличие от **WHERE**, который фильтрует строки до агрегации.

Средняя нагрузка на преподавателя

Query

Query History

1

SELECT

2

ROUND(AVG(занятий), 2) AS среднее_количество_занятий

3

FROM (

4

SELECT

5

teacher_id,

6

COUNT(lesson_id) AS занятий

7

FROM lessons

8

GROUP BY teacher_id

9

) AS subquery;

Data Output

Messages

Notifications

≡+

▼

▼

SQL

среднее_количество_занятий

numeric

1

1.67

Список групп и специальностей (без дубликатов)

Query

Query History

1

SELECT DISTINCT

2

specialty AS специальность

3

FROM groups

4

ORDER BY specialty;

Data Output

Messages

Notifications

≡+

▼

▼

SQL

	специальность character varying (100)
1	Информационные сис...
2	Программирование
3	Тестовая специальнос...
4	Экономика

Студенты и их группы (с использованием подзапроса)

Query

Query History

1

SELECT

2

last_name || ' ' || first_name AS ФИО,

3

(SELECT group_name FROM groups WHERE group_id = s.group_id) AS группа

4

FROM students s

5

ORDER BY last_name;

Data Output

Messages

Notifications

≡+

▼

▼

SQL

Showing

	ФИО text	группа character varying (20)
1	Иванов Иван	П-21
2	Кузнецов Макс...	ИС-22
3	Петров Алексей	П-21
4	Сидорова Анна	ИС-22
5	Смирнова Елена	ЭК-21

Проверка ограничений целостности (практические эксперименты)

Query Query History

```
1 -- Попробуем вставить группу с существующим group_id
2 INSERT INTO groups (group_id, group_name, specialty, admission_year)
3 VALUES (1, 'Новая группа', 'Тест', 2026);
```

Data Output Messages Notifications

ERROR: duplicate key value violates unique constraint "groups_pkey"
Key (group_id)=(1) already exists.

ОШИБКА: duplicate key value violates unique constraint "groups_pkey"
SQL state: 23505
Detail: Key (group_id)=(1) already exists.

Query Query History

```
1 -- Попробуем вставить студента без фамилии
2 INSERT INTO students (first_name, birth_date, group_id)
3 VALUES ('Иван', '2007-01-01', 1);
```

Data Output Messages Notifications

ERROR: значение NULL в столбце "last_name" отношения "students" нарушает ограничение NOT NULL
Ошибочная строка содержит (7, null, Иван, null, 2007-01-01, 1).

ОШИБКА: значение NULL в столбце "last_name" отношения "students" нарушает ограничение NOT NULL
SQL state: 23502
Detail: Ошибочная строка содержит (7, null, Иван, null, 2007-01-01, 1).

Query Query History

```
1 -- Изменим тип данных для столбца middle_name (увеличим длину)
2 ALTER TABLE students ALTER COLUMN middle_name TYPE VARCHAR(100);
```

Data Output Messages Notifications

ALTER TABLE

Query returned successfully in 88 msec.

Query Query History

```
1 -- Попробуем вставить группу с существующим названием
2 INSERT INTO groups (group_name, specialty, admission_year)
3 VALUES ('П-21', 'Другая специальность', 2026);
```

Data Output Messages Notifications

ERROR: повторяющееся значение ключа нарушает ограничение уникальности "groups_group_name_key"
Ключ "(group_name)=(П-21)" уже существует.

ОШИБКА: повторяющееся значение ключа нарушает ограничение уникальности "groups_group_name_key"
SQL state: 23505

Detail: Ключ "(group_name)=(П-21)" уже существует.

Query Query History

```
1  -- Переименуем столбец department в teachers
2  ALTER TABLE teachers RENAME COLUMN department TO kafedra;
3
4  -- Вернём обратно
5  ALTER TABLE teachers RENAME COLUMN kafedra TO department;
```

Data Output Messages Notifications

ALTER TABLE

Query returned successfully in 56 msec.

Query Query History

```
1  -- Добавим временный столбец
2  ALTER TABLE teachers ADD COLUMN temp_column VARCHAR(50);
3
4  -- Удалим его
5  ALTER TABLE teachers DROP COLUMN temp_column;
```

Data Output Messages Notifications

ALTER TABLE

Query returned successfully in 60 msec.

```
Query  Query History
1  -- Добавим ограничение CHECK на год поступления
2  ALTER TABLE groups ADD CONSTRAINT chk_admission_year
3  CHECK (admission_year BETWEEN 2000 AND 2100);
```

Data Output Messages Notifications

```
ALTER TABLE

Query returned successfully in 107 msec.
```

Query

Query History

1

BEGIN TRANSACTION;

2

3

-- Пытаемся удалить всех студентов

4

DELETE FROM students;

5

6

-- Проверяем (внутри транзакции)

7

SELECT COUNT(*) FROM students; -- 0

8

9

-- Откатываем изменения

10

ROLLBACK;

11

12

-- Проверяем после отката

13

SELECT COUNT(*) FROM students; -- 5 (данные восстановлены)

Data Output

Messages

Notifications

≡

📄

▼

📋

▼

🗑

🗄

⬇

⌵

SQL

S

	count
1	5

Query

Query History

1

BEGIN TRANSACTION;

2

3

INSERT INTO groups (group_name, specialty, admission_year)

4

VALUES ('ВРЕМ-01', 'Временная', 2026);

5

6

SAVEPOINT sp1;

7

8

INSERT INTO groups (group_name, specialty, admission_year)

9

VALUES ('ВРЕМ-02', 'Временная', 2026);

10

11

-- Откатываемся только к SAVEPOINT (ВРЕМ-02 не будет добавлена)

12

ROLLBACK TO SAVEPOINT sp1;

13

14

-- Фиксируем (ВРЕМ-01 останется)

15

COMMIT;

16

Data Output

Messages

Notifications

≡

📄

▼

📋

▼

🗑

🗄

⬇

⌵

SQL

Show

	group_id [PK] integer	group_name character varying (20)	specialty character varying (100)	admission_year integer
1	7	ВРЕМ-01	Временная	2026

Создание представлений (VIEW)

Представления — это виртуальные таблицы, которые не хранят данные, а формируются на лету из запроса. Они удобны для упрощения сложных запросов и для разграничения доступа.

Query	Query History
-------	---------------

```
1 CREATE VIEW v_students_with_groups AS
2 SELECT
3     s.student_id,
4     s.last_name || ' ' || s.first_name AS full_name,
5     s.birth_date,
6     g.group_name,
7     g.specialty
8 FROM students s
9 JOIN groups g ON s.group_id = g.group_id;
```

Data Output	Messages	Notifications
-------------	----------	---------------

```
CREATE VIEW

Query returned successfully in 79 msec.
```

Query Query History

```
1 SELECT * FROM v_students_with_groups WHERE group_name = 'П-21';
```

Data Output Messages Notifications

									SQL
	student_id integer		full_name text		birth_date date		group_name character varying (20)		specialty character varying (100)

Query

Query History

1

DROP VIEW v_students_with_groups;

Data Output

Messages

Notifications

DROP VIEW

Query returned successfully in 70 msec.

Создание индексов

Индексы — это структуры, которые ускоряют поиск данных. Они работают как алфавитный указатель в книге. Однако индексы замедляют вставку и обновление, поэтому их нужно создавать обдуманно.

```
1  -- Индекс по фамилии студента (ускоряет поиск по фамилии)
2  CREATE INDEX idx_students_last_name ON students(last_name);
3
4  -- Индекс по дате занятия (ускоряет поиск по дате)
5  CREATE INDEX idx_lessons_date ON lessons(lesson_date);
```

```
CREATE INDEX
```

Query returned successfully in 129 msec.

```
1 EXPLAIN ANALYZE
2 SELECT * FROM students WHERE last_name = 'Иванов';
```

+

📄

▼

📋

▼

🗑️

🗄️

📶

📈

SQL

QUERY PLAN

text

1

Seq Scan on students (cost=0.00..1.06 rows=1 width=466) (actual time=0.014..0.014 rows=1.00 loop...

2

Filter: ((last_name)::text = 'Иванов'::text)

Query

Query History

1

`DROP INDEX idx_students_last_name;`

Data Output

Messages

Notifications

DROP INDEX

Query returned successfully in 77 msec.

Создайте таблицу «Оценки» (grades)

Query Query History

```
1 CREATE TABLE grades (  
2     grade_id SERIAL PRIMARY KEY,  
3     student_id INT NOT NULL,  
4     discipline_id INT NOT NULL,  
5     grade INT NOT NULL CHECK (grade BETWEEN 2 AND 5),  
6     grade_date DATE NOT NULL,  
7  
8     CONSTRAINT fk_grades_students  
9         FOREIGN KEY (student_id)  
10        REFERENCES students(student_id)  
11        ON UPDATE CASCADE  
12        ON DELETE CASCADE,  
13  
14    CONSTRAINT fk_grades_disciplines  
15        FOREIGN KEY (discipline_id)  
16        REFERENCES disciplines(discipline_id)  
17        ON UPDATE CASCADE  
18        ON DELETE RESTRICT  
19 );
```

Data Output Messages Notifications

CREATE TABLE

Query Query History

```
1 INSERT INTO grades (student_id, discipline_id, grade, grade_date) VALUES  
2     (1, 1, 5, '2026-09-15'),  
3     (1, 3, 4, '2026-09-16'),  
4     (2, 1, 3, '2026-09-15'),  
5     (2, 3, 5, '2026-09-16'),  
6     (3, 1, 4, '2026-09-17'),  
7     (3, 2, 5, '2026-09-18'),  
8     (4, 2, 4, '2026-09-18'),  
9     (4, 1, 3, '2026-09-19'),  
10    (5, 2, 5, '2026-09-20'),  
11    (5, 1, 4, '2026-09-21');
```

Data Output Messages Notifications

INSERT 0 10

Аналитические запросы по оценкам

QueryQuery History

1SELECT

2s.last_name || ' ' || s.first_name AS студент,

3ROUND(AVG(g.grade), 2) AS средний_балл

4FROM grades g

5JOIN students s ON g.student_id = s.student_id

6GROUP BY s.student_id, s.last_name, s.first_name

7ORDER BY средний_балл DESC;

Data OutputMessagesNotifications

≡+

▼

▼

SQL

	студент text	средний_балл numeric
1	Иванов Иван	4.50
2	Сидорова Анна	4.50
3	Смирнова Елена	4.50
4	Петров Алексей	4.00
5	Кузнецов Макс...	3.50

Query Query History

```

1  SELECT
2      d.discipline_name AS дисциплина,
3      ROUND(AVG(g.grade), 2) AS средний_балл,
4      COUNT(g.grade_id) AS количество_оценок
5  FROM grades g
6  JOIN disciplines d ON g.discipline_id = d.discipline_id
7  GROUP BY d.discipline_id, d.discipline_name
8  ORDER BY средний_балл DESC;

```

Data Output Messages Notifications

	дисциплина character varying (100)	средний_балл numeric	количество_оценок bigint
1	Информационные технологии	4.67	3
2	Программирование	4.50	2
3	Основы проектирования баз данн...	3.80	5

Query Query History

```
1 SELECT
2     s.last_name || ' ' || s.first_name AS студент,
3     d.discipline_name AS дисциплина,
4     g.grade AS оценка,
5     g.grade_date AS дата
6 FROM grades g
7 JOIN students s ON g.student_id = s.student_id
8 JOIN disciplines d ON g.discipline_id = d.discipline_id
9 WHERE g.grade < 4
10 ORDER BY g.grade, s.last_name;
```

Data Output Messages Notifications

	студент text	дисциплина character varying (100)	оценка integer	дата date
1	Кузнецов Макс...	Основы проектирования баз данн...	3	2026-09-...
2	Петров Алексей	Основы проектирования баз данн...	3	2026-09-...

Лучшие студенты (средний балл > 4.5)

Сделать самостоятельно.

Триггеры (автоматизация действий)

Триггеры — это специальные функции, которые автоматически выполняются при наступлении определённого события (вставка, обновление, удаление). Они позволяют автоматизировать поддержание целостности и ведение аудита.

Query Query History

```
1 CREATE TABLE audit_log (  
2     log_id SERIAL PRIMARY KEY,  
3     table_name VARCHAR(50),  
4     operation VARCHAR(10),  
5     record_id INT,  
6     old_data JSONB,  
7     new_data JSONB,  
8     changed_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
9     changed_by VARCHAR(50) DEFAULT CURRENT_USER  
10 );
```

Data Output Messages Notifications

CREATE TABLE

Query returned successfully in 86 msec.

Query Query History

```
1 CREATE OR REPLACE FUNCTION audit_students_changes()
2 RETURNS TRIGGER AS $$
3 BEGIN
4     IF TG_OP = 'INSERT' THEN
5         INSERT INTO audit_log (table_name, operation, record_id, new_data)
6         VALUES ('students', 'INSERT', NEW.student_id, to_jsonb(NEW));
7         RETURN NEW;
8     ELSIF TG_OP = 'UPDATE' THEN
9         INSERT INTO audit_log (table_name, operation, record_id, old_data, new_data)
10        VALUES ('students', 'UPDATE', NEW.student_id, to_jsonb(OLD), to_jsonb(NEW));
11        RETURN NEW;
12    ELSIF TG_OP = 'DELETE' THEN
13        INSERT INTO audit_log (table_name, operation, record_id, old_data)
14        VALUES ('students', 'DELETE', OLD.student_id, to_jsonb(OLD));
15        RETURN OLD;
16    END IF;
17 END;
18 $$ LANGUAGE plpgsql;
19
20 CREATE TRIGGER trg_students_audit
21 AFTER INSERT OR UPDATE OR DELETE ON students
22 FOR EACH ROW EXECUTE FUNCTION audit_students_changes();
```

Data Output Messages Notifications

CREATE TRIGGER

Query returned successfully in 61 msec.

Query Query History

```
1  -- Вставляем студента
2  INSERT INTO students (last_name, first_name, birth_date, group_id)
3  VALUES ('Новиков', 'Пётр', '2007-05-05', 1);
4
5  -- Обновляем
6  UPDATE students SET first_name = 'Петр' WHERE last_name = 'Новиков';
7
8  -- Удаляем
9  DELETE FROM students WHERE last_name = 'Новиков';
10
11 -- Смотрим аудит
12 SELECT * FROM audit_log ORDER BY changed_at;
```

Data Output Messages Notifications

≡+

▼

▼

SQL

Showing rows: 1 to 3

	log_id [PK] integer	table_name character varying (50)	operation character varying (10)	record_id integer	old_data jsonb
1	1	students	INSERT	8	[null]
2	2	students	UPDATE	8	{"group_id": 1, "last_name": "Новиков", "birth_date": "2007-05-05", "first_name": "Пётр", "group_id": 1}
3	3	students	DELETE	8	{"group_id": 1, "last_name": "Новиков", "birth_date": "2007-05-05", "first_name": "Пётр", "group_id": 1}

Query

Query History

1

CREATE OR REPLACE FUNCTION check_grade_range()

2

RETURNS TRIGGER AS \$\$

3

BEGIN

4

IF NEW.grade < 2 OR NEW.grade > 5 THEN

5

RAISE EXCEPTION 'Оценка должна быть от 2 до 5, получено: %', NEW.grade;

6

END IF;

7

RETURN NEW;

8

END;

9

\$\$ LANGUAGE plpgsql;

10

11

CREATE TRIGGER trg_check_grade

12

BEFORE INSERT OR UPDATE ON grades

13

FOR EACH ROW EXECUTE FUNCTION check_grade_range();

Data Output

Messages

Notifications

CREATE TRIGGER

Query returned successfully in 133 msec.

Разграничение прав доступа (DCL)

В реальных системах разные пользователи должны иметь разные права. Администратор может всё, преподаватель — видеть своих студентов, студент — только свои оценки. Научимся управлять правами.

Создание ролей и пользователей

Query

Query History

1

-- Создаём роли

2

CREATE ROLE teacher_role;

3

CREATE ROLE student_role;

4

5

-- Создаём пользователей

6

CREATE USER teacher_ivanov WITH PASSWORD 'secure_password_1';

7

CREATE USER student_petrov WITH PASSWORD 'secure_password_2';

8

9

-- Назначаем роли

10

GRANT teacher_role TO teacher_ivanov;

11

GRANT student_role TO student_petrov;

Data Output

Messages

Notifications

GRANT ROLE

Query returned successfully in 53 msec.

Предоставление прав

Query	Query History
1	-- Преподаватель может читать студентов и занятия
2	GRANT SELECT ON students TO teacher_role;
3	GRANT SELECT ON lessons TO teacher_role;
4	GRANT SELECT ON groups TO teacher_role;
5	GRANT SELECT ON disciplines TO teacher_role;
6	
7	-- Студент может читать только свои оценки (через представление)
8	CREATE VIEW v_my_grades AS
9	SELECT
10	gr.grade_id,
11	s.student_id,
12	d.discipline_name,
13	gr.grade,
14	gr.grade_date
15	FROM grades gr
16	JOIN students s ON gr.student_id = s.student_id
17	JOIN disciplines d ON gr.discipline_id = d.discipline_id;
18	
19	GRANT SELECT ON v_my_grades TO student_role;

Data Output	Messages	Notifications
GRANT		
Query returned successfully in 60 msec.		